Interfaces graphiques avec JavaFX

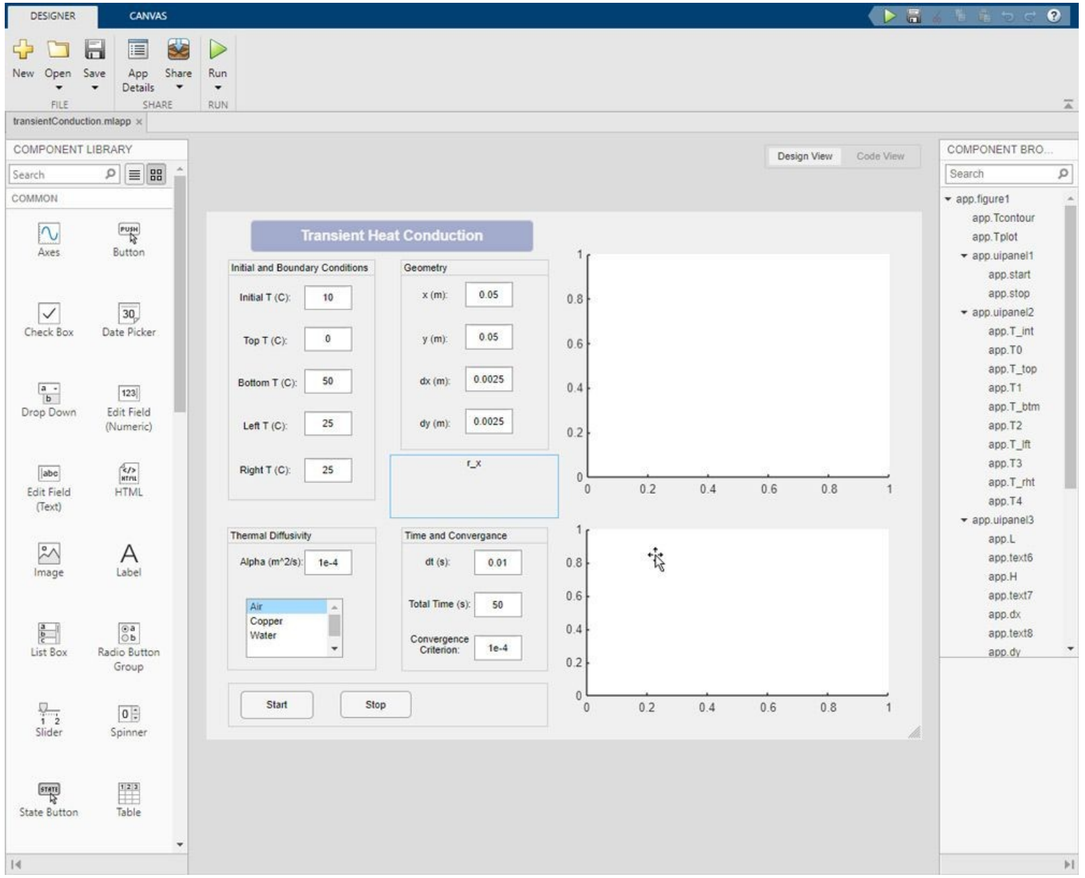
Dany Siriphol – CODA 1 ère année - 2025

Présentation de JavaFX

Toolkit graphique permettant de créer des applications de desktop

Disponible depuis Java 8
Remplace Swing et AWT

- Grande librairie de composant
- Gestion d’evenements
- Support multimédia : Image, Vidéo, Son
- Style avec CSS
- Possibilité de construire une interface avec un langage similaire au HTML : FXML
- Gère l’affichage 3D



Installation de JavaFX

Installation de JavaFX

JavaFX ne fait pas partie du SDK de base et il faut l’installer à part sur les projets

Télécharger JavaFX sur le site de Gluon

- https://gluonhq.com/products/javafx/

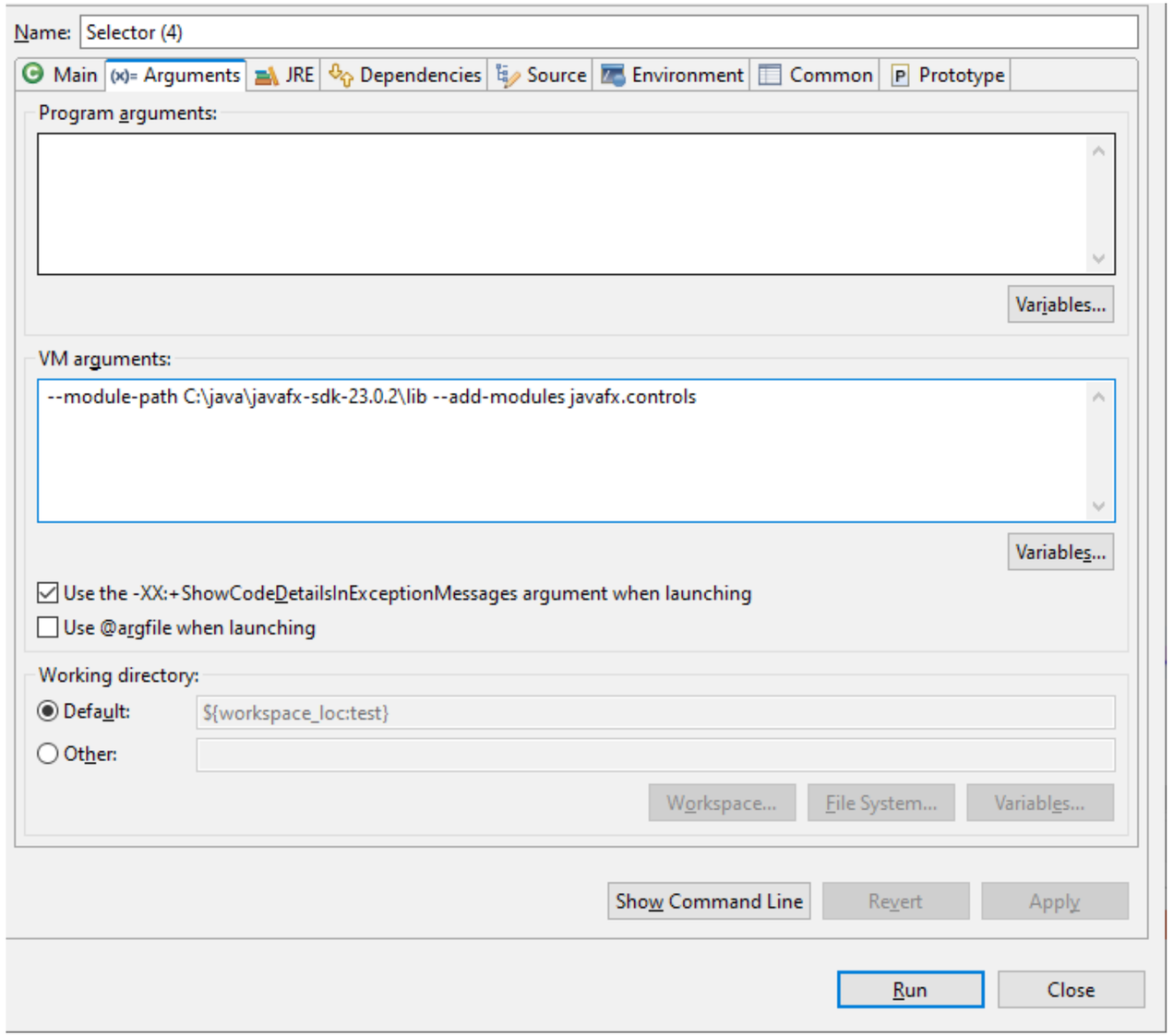
Ajouter les .jar de JavaFX au build path de votre IDE

- https://openjfx.io/openjfx-docs/

Installation de JavaFX

Modifier la commande de compilation de votre projet pour lui ajouter ces paramètres

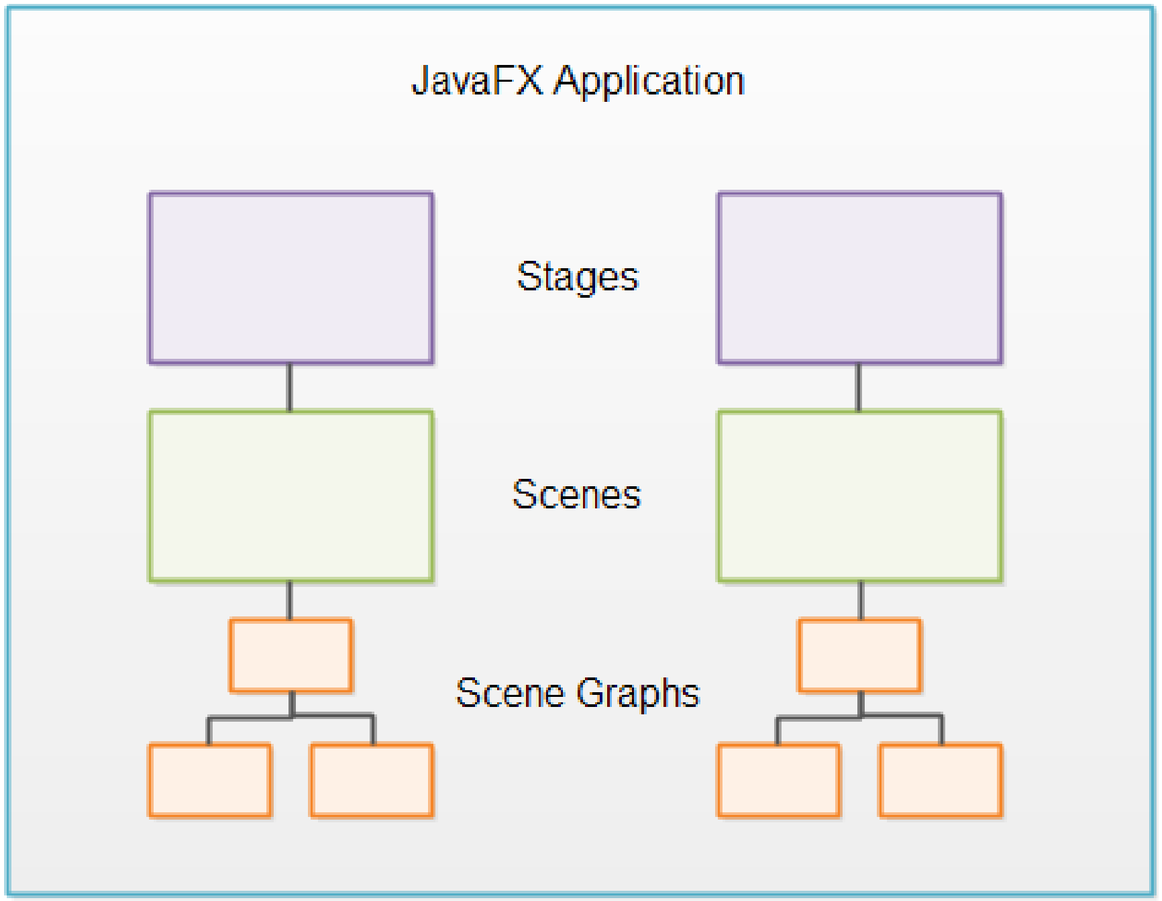
--module-path C:\java\javafx-sdk-21.0.6\lib --add-modules javafx.controls,javafx.fxml



Structure d’une application JavaFX

Dany Siriphol – CODA 1 ère année - 2025

Structure d’une application JavaFX



Présentation de JavaFX

```
public class MyApplication extends Application {

    public void start(Stage stage) {
        StackPane root = new StackPane();
        Scene scene = new Scene(root, 500, 500);
        stage.setScene(scene);
        stage.setTitle("Hello World");
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

Structure d’une application JavaFX

Stage

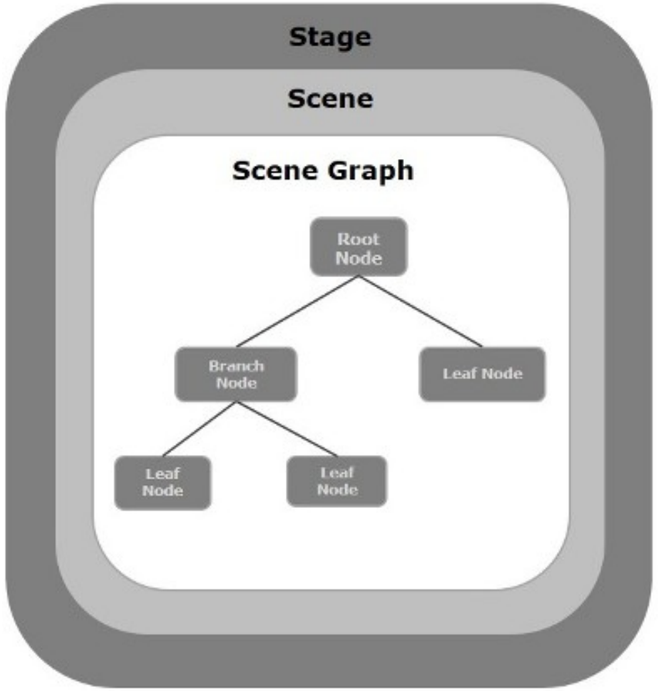
Représente la fenêtre de votre application

Scene

Chaque Stage doit contenir une Scene. La scene est un container qui contient tout les éléments de votre application

Nodes

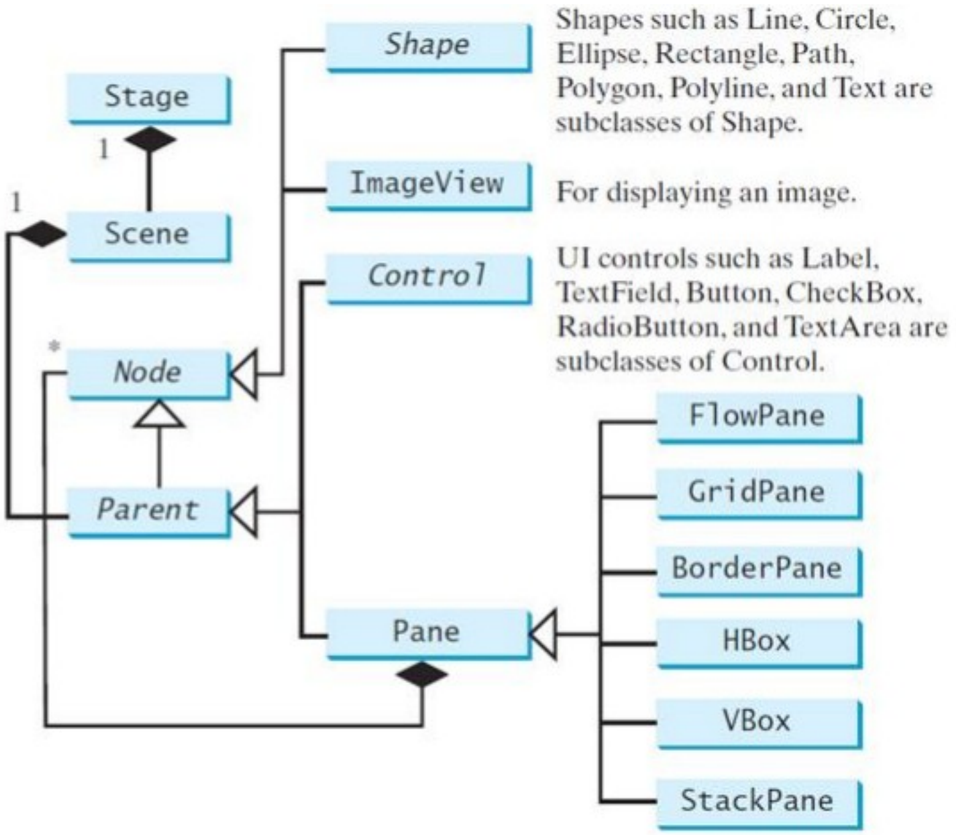
Une application JavaFX est composé d’un ensemble d’élements (Node) organisé sous la forme d’un graphe. Ces nodes peuvent être des éléments d’UI ou des containers



Composants JavaFX

Dany Siriphol – CODA 1 ère année - 2025

Hierarchie des composants JavaFX



JavaFX : Controls

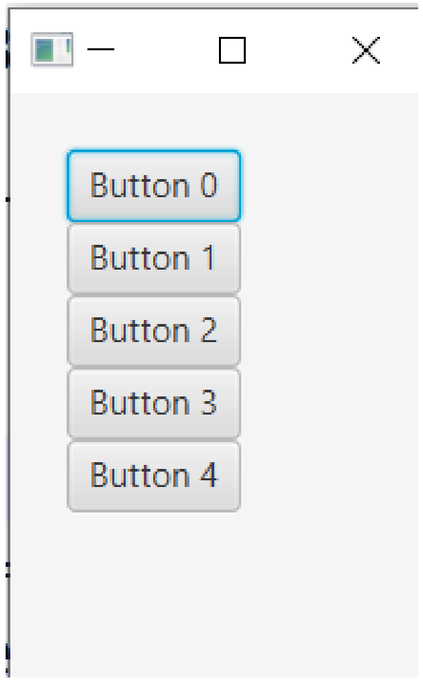
Accordion
Button
CheckBox
ChoiceBox
ColorPicker
ComboBox
DatePicker
Label
ListView
Menu
MenuBar
PasswordField
ProgressBar
RadioButton
Slider
Spinner
SplitMenuButton
SplitPane
TableView
TabPane
TextArea
TextField
TitledPane
ToggleButton
ToolBar
TreeTableView
TreeView

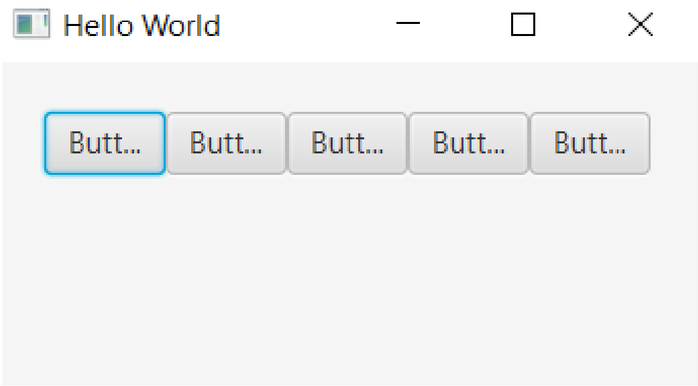
JavaFX : Layouts

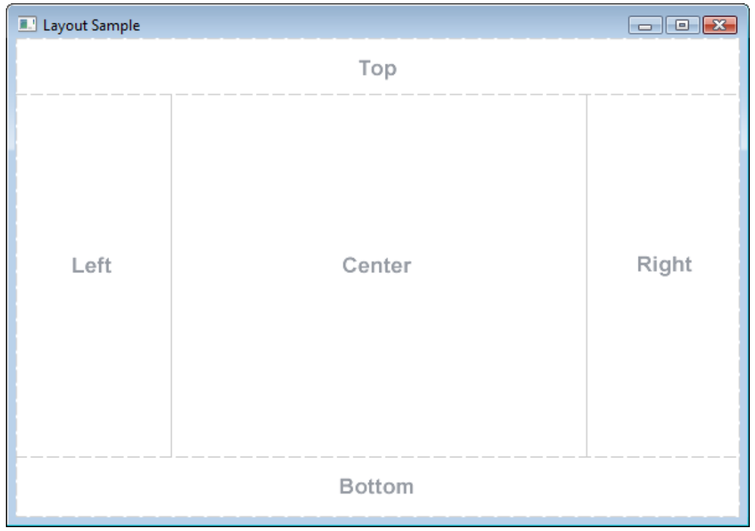
Pour gérer le positionnement des éléments, JavaFX utilise des composants qui servent de « containers » pour d’autres éléments et qui vont chacun avoir une gestion de flux particulière (Layout)

Group
Region
Pane
HBox
VBox
FlowPane
BorderPane
BorderPane
StackPane
TilePane
GridPane
AnchorPane
TextFlow

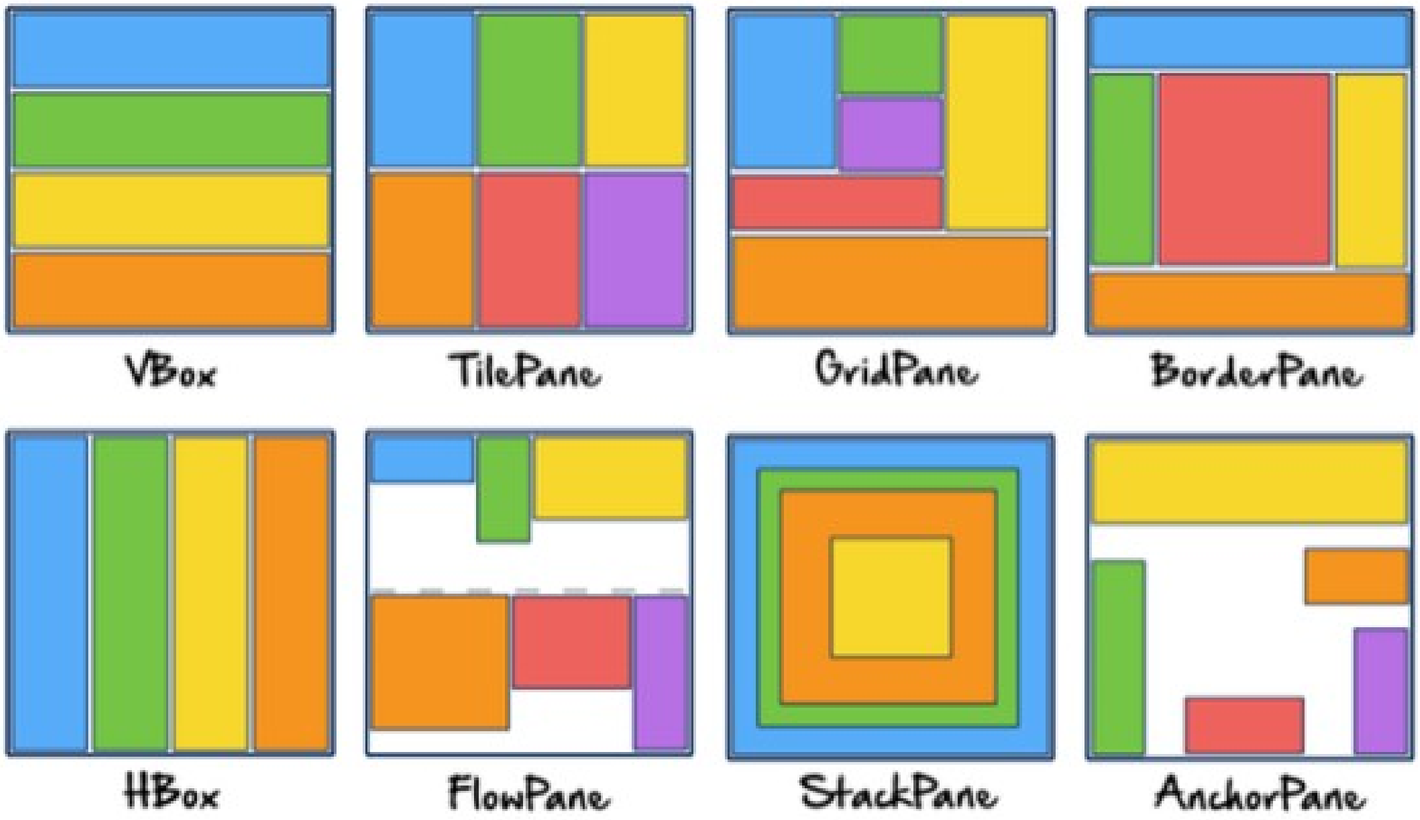
JavaFX : Layouts





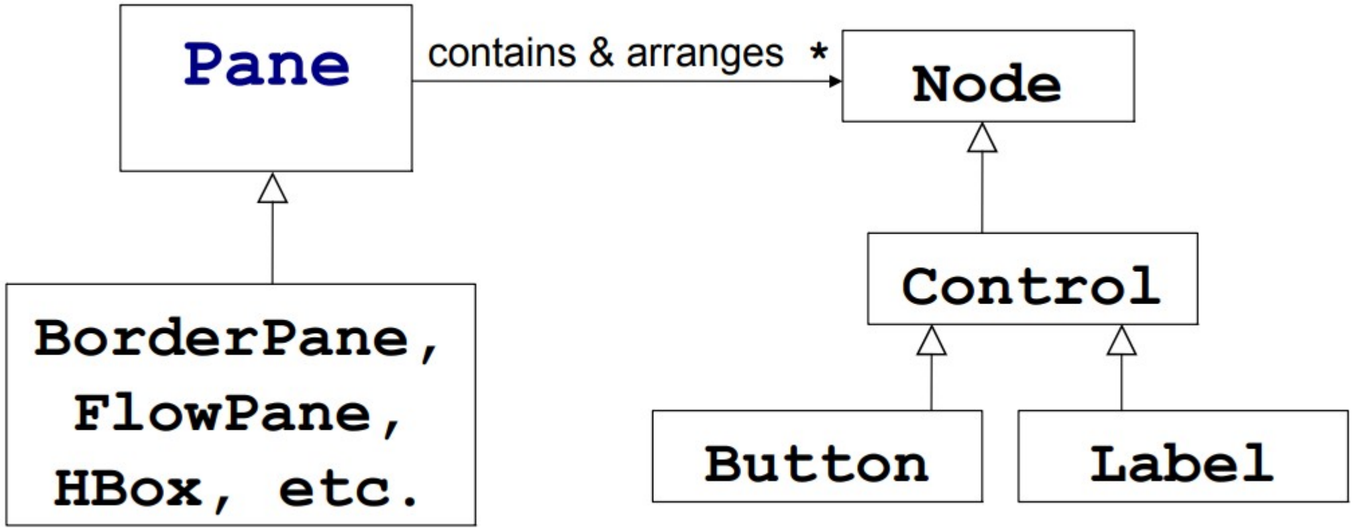


Containers

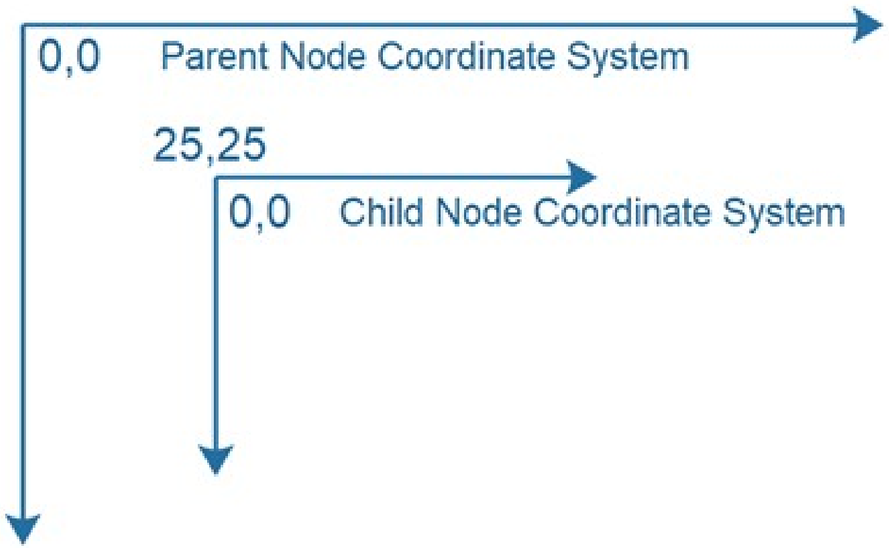

VBox
TilePane
GridPane
BorderPane
HBox
FlowPane
StackPane
AnchorPane

Ajout d’éléments dans un container

```
StackPane pane = new StackPane();
pane.getChildren().add(new Button("OK"));
```



Coordonnées : Positionnement absolu




```
Button b = new Button("OK");
b.setLayoutX(100);
b.setLayoutY(200);

Pane pane = new Pane();
pane.getChildren().add(b);
```

Gestion d’évènements

Dany Siriphol – CODA 1 ère année - 2025

Programmation évènementielle

Faire réagir votre application lors d’un evenement:

- Souris
- Clavier
- Système

Pour cela il faut attacher des « écouteurs d’évènements » sur les éléments de votre interface qui vont lancer une **fonction (event handler)** lorsque l’évènement est déclenché

Poser un écouteur d’evenements

Pour poser un écouteur d’evenements, il faut attacher une méthode à un Node, il existe 2 façons :

Utiliser la méthode addEventHandler

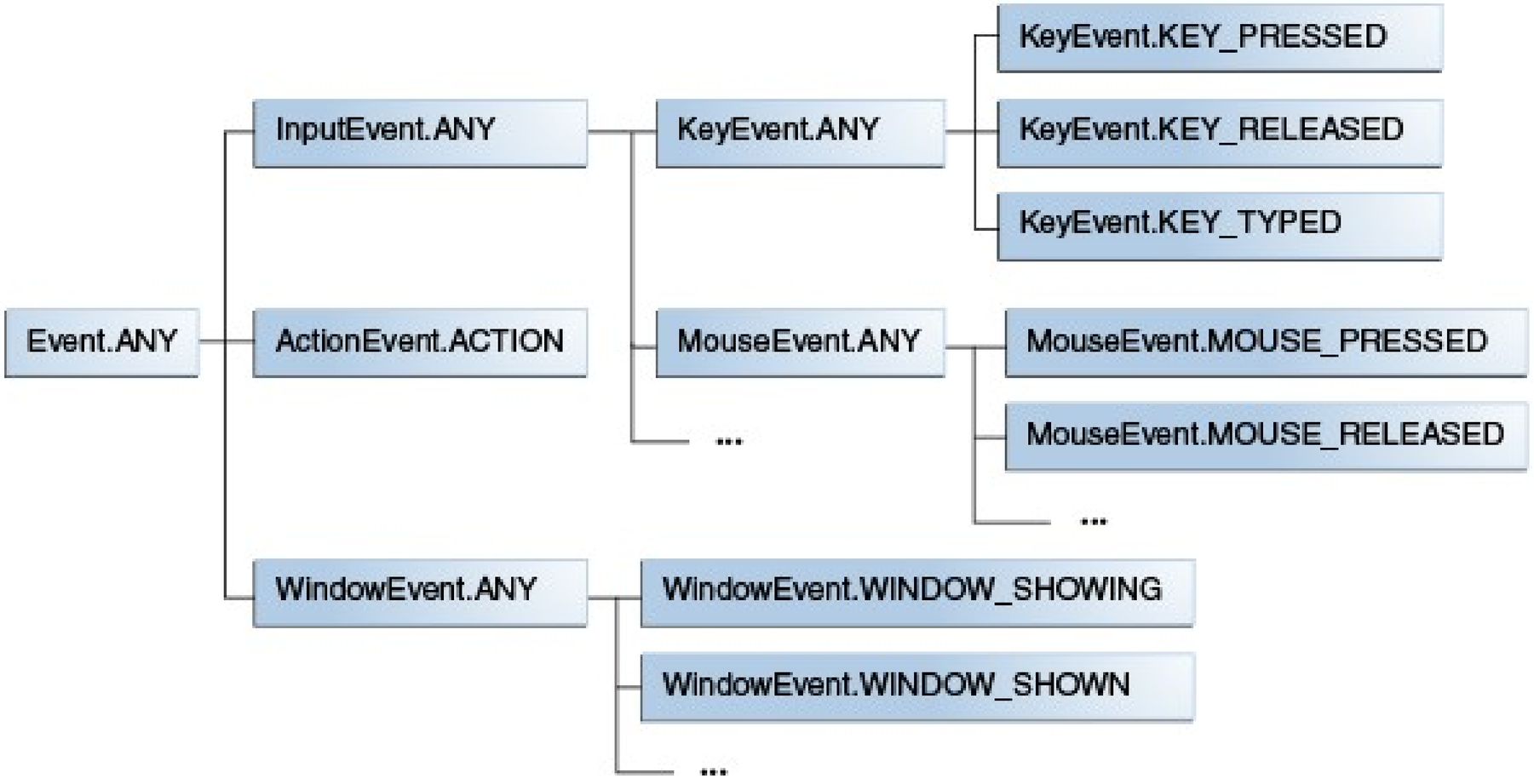
Utiliser une méthode spécifique

- setOnAction
- SetOnMouseClicked
- …

Il faut ensuite spécifier la méthode qui va servir d’event handler

```
button.addEventHandler(ActionEvent.ANY, new ButtonHandler( ) );
```

Type d’evenements



Ajouter un EventHandler

Classe anonyme

```
button.setOnMouseClicked(new EventHandler<MouseEvent>() {
    @Override
    public void handle(MouseEvent e) {
        System.out.println("Button clicked!");
    }
});
```

Ajouter un EventHandler

Classe externe

```java
import javafx.event.ActionEvent;
import javafx.event.EventHandler;

public class ClickHandler implements EventHandler<ActionEvent> {
    @Override
    public void handle(ActionEvent e) {
        System.out.println("External class");
    }
}
```

```java
button.addEventHandler(ActionEvent.ANY, new ClickHandler( ) );
```

Ajouter un EventHandler

Méthode statique (référence de méthode)

```
private static void handleButtonClick(ActionEvent e) {
    System.out.println("Button clicked!");
}
```

```
b.setOnAction(MyApplication::handleButtonClick);
```

Ajouter un EventHandler

Lambda expression

```
button.setOnMouseClicked((event) -> {
    System.out.println("Button clicked!");
});
```

Exercices